

VANIR · SECURITY REVIEW

Vanir Review.

A security review of Review.

CLIENT	PROJECT	AUDITORS	AUDIT WINDOW	FINAL REPORT
Vanir	Review	Guardian	November 28 – February 20, 2025	February 27, 2025

§ 00

Contents.

60 pages — overview, confidence ranking, scope, 46 findings, disclaimer, and about Guardian.

§ 01	Overview	P. 03
§ 02	Confidence Ranking	P. 04
§ 03	Scope & Classifications	P. 05
	Files in Scope	P. 06
§ 04	Findings	P. 07
	Main Review · 26 findings	P. 11
	Remediation Review 1 · 8 findings	P. 37
	Remediation Review 2 · 9 findings	P. 47
	Remediation Review 3 · 3 findings	P. 56
§ 05	Disclaimer	P. 59
§ 06	About Guardian	P. 60

§ 01

Overview.

Guardian reviewed Review across 4 review rounds. The final report includes 1 Critical, 6 High, 6 Medium, 33 Low findings.

AUDIT FIRM	REPOSITORY
Guardian	MEWB-AS/vanir-contracts
CLIENT	METHODOLOGY
Vanir	Smart contract · Manual · Main Review · Remediation Review 1 · Remediation Review 2 · Remediation Review 3
PROJECT	MAIN REVIEW COMMIT
Review	2f037ffc3889
PREPARED BY	REMEDATION REVIEW 1 COMMIT
Guardian	765f91c3544c
AUDIT WINDOW	REMEDATION REVIEW 2 COMMIT
November 28 – February 20, 2025	fdef6e1da61d
FINAL REPORT	REMEDATION REVIEW 3 COMMIT
February 27, 2025	0a0e05b49e65

1.1 Confidence Ranking

2 / 5

Low Confidence

Guardian assigns a Confidence Ranking of 2 out of 5, combining finding severity, remediation outcome, and the reviewer’s assessment of latent risk. The full scale is on the next page.

1	2	3	4	5
VERY LOW CONFIDENCE	LOW CONFIDENCE	MODERATE CONFIDENCE	HIGH CONFIDENCE	VERY HIGH CONFIDENCE

1.2 Findings at a Glance

SEVERITY	TOTAL	PENDING	ACK	PARTIAL	RESOLVED
● Critical	1	0	0	0	1
● High	6	0	2	0	4
● Medium	6	0	3	0	3
● Low	33	0	14	0	19
● Informational	0	0	0	0	0
All severities	46	0	19	0	27

§ 02

Confidence Ranking.

Guardian assigns a project-level confidence score using the scale below. The ranking combines finding severity, remediation outcome, and the reviewer's assessment of latent risk.

CONFIDENCE RANKING	DEFINITION AND RECOMMENDATION	RISK PROFILE
5 Very High Confidence	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: code is highly secure at time of audit.	0 High/Critical findings and few Low/Medium severity findings.
4 High Confidence	Code is clean, well-structured, and adheres to best practices. Design patterns are sound and test coverage is strong. Recommendation: suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity findings.
3 Moderate Confidence	Code is functional but shows gaps in defensive programming or test coverage. Recommendation: remediate the findings and re-review the changes before deployment.	Some High/Critical findings, with various Medium severity findings.
2 Low Confidence ASSIGNED	Recurring issues suggest systemic gaps rather than isolated mistakes. Recommendation: remediate, then commission a follow-up review at a frozen commit.	Over 6 High and Critical findings across the engagement.
1 Very Low Confidence	The volume and severity of findings indicate the codebase is not ready for production. Recommendation: substantial rework, then a full re-review.	Over 12 High and Critical findings across the engagement.

§ 03

Scope & Classifications.

Guardian reviewed Review in `MEWB-AS/vanir-contracts`. The scope includes 150 source lines across 2 files.

3.1 Severity Matrix

	IMP · HIGH	IMP · MED	IMP · LOW
LIK · HIGH	Critical	High	Medium
LIK · MED	High	Medium	Low
LIK · LOW	Medium	Low	Low

IMPACT · HIGH

Significant loss of assets, harm to a group of users, or core protocol functionality is disrupted.

IMPACT · MEDIUM

Small amount of funds can be lost, or ancillary functionality is affected. Users may experience reduced or delayed receipt.

IMPACT · LOW

Unexpected behavior in some functionality that is notable but does not meet the criteria for higher severity.

LIKELIHOOD · HIGH

Possible with reasonable assumptions that mimic on-chain conditions; cost is low compared to amount gained.

LIKELIHOOD · MEDIUM

Possible only in uncommon cases or requires a large amount of capital relative to disruption caused.

LIKELIHOOD · LOW

Unlikely to ever occur in production.

Files in Scope.

FILE	NSLOC	TOTAL
src/Vanir.sol	124	176
src/Interfaces.sol	26	28

§ 04

Findings.

Grouped by round. Visible IDs reset by severity within each round, so a finding is named the same way in the report as it was in the channel it was written in.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
MAIN REVIEW					
H-01	Liquidations/Withdrawals Might Revert If The User Can Not Receive Native Assets	Logical Error	HIGH	ACKNOWLEDGED	p. 11
H-02	Liquidation Calls Can Be Sandwiched	Logical Error	HIGH	ACKNOWLEDGED	p. 12
H-03	Liquidation Calls Can Be DoS'd	DoS	HIGH	RESOLVED	p. 13
H-04	Liquidation Fee Is Wrongly Sent To The User	Configuration	HIGH	RESOLVED	p. 14
M-01	Loan Function Does Not Support Multiple Requests With Native Ether	Configuration	MEDIUM	ACKNOWLEDGED	p. 15
M-02	USDT Actions Are Blocked After First Withdrawal	DoS	MEDIUM	RESOLVED	p. 16
L-01	State Variables Missing Immutable Modifier	Optimization	LOW	RESOLVED	p. 17
L-02	getUserAccountData Function Can Be Declared As View	Optimization	LOW	RESOLVED	p. 18
L-03	code Parameter Can Be Removed From The loan Function	Optimization	LOW	RESOLVED	p. 19
L-04	USDS Liquidations Are Not Supported	Optimization	LOW	RESOLVED	p. 20
L-05	Liquidations Require A SPToken Approval To The Vanir Contract	Logical Error	LOW	RESOLVED	p. 21

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
L-06	Use Of Transfer Instead Of Call	Configuration	LOW	ACKNOWLEDGED	p. 22
L-07	Owner State Variable Can Be Removed	Optimization	LOW	RESOLVED	p. 23
L-08	Insecure setAdmin Function	Configuration	LOW	RESOLVED	p. 24
L-09	Unlicensed Smart Contract	Configuration	LOW	ACKNOWLEDGED	p. 25
L-10	Decreased Health Factor After Liquidation	Logical Error	LOW	RESOLVED	p. 26
L-11	Maximum Withdraw Is Not Supported	Configuration	LOW	RESOLVED	p. 27
L-12	Unused fee Parameter	Optimization	LOW	RESOLVED	p. 28
L-13	Users Can Escape The Liquidation Fee	Configuration	LOW	ACKNOWLEDGED	p. 29
L-14	Unnecessary Token Approval In withdraw	Logical Error	LOW	RESOLVED	p. 30
L-15	Unused fallback Function	Optimization	LOW	RESOLVED	p. 31
L-16	Referral Code Ommited	Configuration	LOW	ACKNOWLEDGED	p. 32
L-17	setAdmin() Should Be external	Optimization	LOW	RESOLVED	p. 33
L-18	Lack Of Reentrancy Guard Modifier	Configuration	LOW	ACKNOWLEDGED	p. 34
L-19	Check Debt Amounts During liquidate	Validation	LOW	RESOLVED	p. 35
L-20	Unvalidated Token Addresses	Validation	LOW	ACKNOWLEDGED	p. 36

REMEDIATION REVIEW 1

C-01	Unvalidated Parameters Allow an Attacker to Drain User Funds	Validation	CRITICAL	RESOLVED	p. 37
H-01	Unrestricted Direct Contract Interaction	Validation	HIGH	RESOLVED	p. 40

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
H-02	Liquidations Will Revert When Flashloan Premium Is Different Than 0	Logical Error	HIGH	RESOLVED	p. 41
M-01	USDT swap is blocked	DoS	MEDIUM	RESOLVED	p. 42
L-01	Owner cannot be admin	Code Best Practices	LOW	RESOLVED	p. 43
L-02	Typo	Code Best Practices	LOW	RESOLVED	p. 44
L-03	Token And spToken Can Mismatch	Validation	LOW	ACKNOWLEDGED	p. 45
L-04	Validate Asset Is Not Used Twice	Validation	LOW	ACKNOWLEDGED	p. 46

REMEDIATION REVIEW 2

M-01	User Can Pass Arbitrary Fee Amount	Logical Error	MEDIUM	ACKNOWLEDGED	p. 47
M-02	Failed Approvals With USDT	Logical Error	MEDIUM	RESOLVED	p. 48
M-03	Lost borrow fee	Validation	MEDIUM	ACKNOWLEDGED	p. 49
L-01	LendingPoolAddress Points To Registry Instead	Deployment	LOW	RESOLVED	p. 50
L-02	No Controls On Liquidation Fee Amount	Validation	LOW	RESOLVED	p. 51
L-03	Failed Liquidations With Low Target LTV	Warning	LOW	RESOLVED	p. 52
L-04	Failing Tests	Warning	LOW	ACKNOWLEDGED	p. 53
L-05	No events emitted on funds rescue	Event	LOW	ACKNOWLEDGED	p. 54
L-06	Frozen Pool Will Lead To Fail Liquidations	Warning	LOW	ACKNOWLEDGED	p. 55

REMEDIATION REVIEW 3

L-01	Incorrected Addresses	Warning	LOW	ACKNOWLEDGED	p. 56
------	-----------------------	---------	-----	--------------	-------

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
L-02	Leftover SwapRouter Approval	Warning	LOW	ACKNOWLEDGED	p. 57
L-03	Overcharge in executeOperation	Logical Error	LOW	ACKNOWLEDGED	p. 58

HIGH **H-01**

ACKNOWLEDGED

Liquidations/Withdrawals Might Revert If The User Can Not Receive Native Assets

CATEGORY Logical Error
LOCATION Vanir.sol: 90
REVIEW Main Review

— DESCRIPTION

The `Vanir` contract's liquidation and withdrawal functions may revert if the user cannot receive native Ether. In the `withdrawEth` and `transferEth` functions, the contract unwraps Wrapped Ether to Ether and attempts to send it to the user's address using `payable(to).transfer(amount)`. This method assumes that the recipient address can accept Ether transfers. However, if the user's address is a smart contract without a payable fallback or receive function, or if it deliberately rejects Ether transfers, the transfer call will fail, causing the entire transaction to revert.

— RECOMMENDATION

Consider creating a wrapper function to send Native assets. If the native transfer fails, consider wrapping the Ether into Wrapped Ether and transferring it to the destination address as an ERC20 token, which should never revert.

— RESOLUTION

Status: Acknowledged.

HIGH **H-02**

ACKNOWLEDGED

Liquidation Calls Can Be Sandwiched

CATEGORY Logical Error
LOCATION Vanir.sol: 170
REVIEW Main Review

DESCRIPTION

The `Vanir` contract is susceptible to sandwich attacks during the liquidation process, specifically when swapping a portion of the collateral tokens for the borrowed tokens in the `swap` function. The liquidation process involves taking a flash loan of the borrowed token (e.g., DAI) to repay the user's debt. It then withdraws the user's collateral (e.g., WETH) by utilizing the user's approved `spWETH` tokens. The contract swaps a portion of this collateral to obtain the exact amount of debt tokens needed to repay the flash loan, using Uniswap's `exactOutput` function. This function allows the input amount (collateral tokens) to vary up to a maximum (`amountInMaximum`), which is the total collateral amount approved by the user. Because the transaction details will be visible in the public mempool, a malicious user could perform a sandwich attack by front-running and manipulating the token reserves unfavorably just before the swap occurs. This manipulation increases the swap's cost, causing the contract to consume more of the user's collateral than necessary. The attacker profits by selling tokens to inflate the price before the swap and buying them back at a lower price afterward. As a result, the user receives less collateral back after liquidation, leading to a loss beyond what is necessary to cover their debt.

This issue can occur whenever the `requests[i].collateral.amount` value is higher than `request.debtAmount` as `requests[i].collateral.amount` will be used as the `amountInMaximum` parameter in the `exactOutput` call.

RECOMMENDATION

Whenever the `liquidate` function is called, consider setting `requests[i].collateral.amount` to a value equal to the expected `amountOut` value needed to repay the flashloan.

RESOLUTION

Status: Acknowledged.

HIGH **H-03**

RESOLVED

Liquidation Calls Can Be DoS'd

CATEGORY DoS
LOCATION Vanir.sol: 166
REVIEW Main Review

DESCRIPTION

The `swap` function includes a `require` statement that compares the contract's balance of the collateral token (`amountInMaximum`) to the expected collateral amount to be liquidated (`request.collateral.amount`):

```
require(amountInMaximum == request.collateral.amount, "vanir/contract balance not equal amount of collateral to be liquidated.");
```

`amountInMaximum` is determined by the contract's current balance of the collateral token:

```
uint amountInMaximum = Token(request.collateral.tokenAddress).balanceOf(address(this));
```

An attacker can exploit this by sending a minimal amount (e.g., 1 wei) of the collateral token to the contract's address before a liquidation occurs. This token transfer increases the contract's balance, causing `amountInMaximum` to exceed `request.collateral.amount`. As a result, the `require` statement fails, and the liquidation transaction reverts. This vulnerability enables any malicious actor to indefinitely prevent liquidations by manipulating the contract's token balance.

RECOMMENDATION

Consider updating the `require` check from the `swap` function as shown below:

```
require(amountInMaximum >= request.collateral.amount, "vanir/contract balance not equal amount of collateral to be liquidated.");
```

RESOLUTION

Status: Resolved.

<https://github.com/MEWB-AS/vanir-contracts/pull/12/commits/fbf77b3e08873ca7f6c23135f8a922e9b858d03f>

HIGH **H-04**

RESOLVED

Liquidation Fee Is Wrongly Sent To The User

CATEGORY Configuration

LOCATION Vanir.sol: 174

REVIEW Main Review

DESCRIPTION

In the `Vanir` contract, during the liquidation process, the liquidation fee specified by `request.collateral.fee` is incorrectly handled and ends up being returned to the user instead of being collected by the protocol or sent to a designated fee address. Specifically, in the `swap` function, after swapping the collateral tokens to repay the user's debt, the remaining collateral is calculated by subtracting only the `swapCost` from `amountInMaximum`:

```
return (amountInMaximum - swapCost, swapCost);
```

This calculation does not account for the liquidation fee (`request.collateral.fee`), meaning that the fee is not deducted from the user's remaining collateral. As a result, when the contract transfers the `colAfterSwap` amount back to the user, it includes the liquidation fee.

RECOMMENDATION

Update the calculation of the remaining collateral to ensure that the liquidation fee is properly deducted before returning funds to the user:

```
amountAfterSwap = amountInMaximum - swapCost - request.collateral.fee;
```

Additionally, implement logic to transfer the collected liquidation fee to a designated fee address.

RESOLUTION

Status: Resolved.

<https://github.com/MEWB-AS/vanir-contracts/pull/12/commits/7e051f8cc8d48ad7788cc49213ed36f32ccb5acd>

MEDIUM **M-01**

ACKNOWLEDGED

Loan Function Does Not Support Multiple Requests With Native Ether

CATEGORY Configuration
LOCATION Vanir.sol: 105
REVIEW Main Review

DESCRIPTION

The `Vanir` contract's `loan` function does not support multiple requests involving native Ether (ETH) within a single transaction. This limitation occurs because the function relies on `msg.value` to determine the amount of Ether supplied but `msg.value` represents the total amount of Ether sent with the transaction, not per individual request. In the loop within the `loan` function, when handling multiple `LoanRequest` items that require supplying Ether (`isEth` is `true`), each call to `supplyEth` expects `msg.value` to equal the `amount` specified for that request:

```
require(msg.value > 0, "vanir/msg value must be larger than.");  
require(msg.value == amount, "vanir/msg value must be the same as amount");
```

Since `msg.value` can not be different for each iteration, these `require` statements will fail if there are multiple Ether requests, as `msg.value` cannot simultaneously equal multiple different `amount` values. Even if the total `msg.value` covers all Ether amounts across requests, the per-request `require` checks will not pass. This means that users cannot perform multiple supply or withdrawal actions involving Ether in a single `loan` function call, limiting the contract's functionality and causing transactions to revert with an "out of funds" error.

RECOMMENDATION

To enable multiple requests involving native Ether within a single `loan` function call, the contract should be updated to handle Ether amounts per request rather than relying solely on `msg.value`. One possible approach is to accumulate the total Ether required for all `LoanRequest` items involving Ether and ensure that `msg.value` equals this total.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-02**

RESOLVED

USDT Actions Are Blocked After First Withdrawal

CATEGORY DoS
LOCATION Vanir.sol: 76
REVIEW Main Review

DESCRIPTION

USDT is not supported as collateral but is supported as an underlying asset in Spark. This means users can deposit USDT to earn interest. Unlike regular ERC-20 tokens, USDT token approvals work differently, and previous allowances must be cleared before granting new approvals.

In the `Vanir.withdraw` function, an approval is granted to the lending pool, but this allowance is never utilized. After the first withdrawal of USDT through Vanir, all subsequent USDT deposits and withdrawals are blocked due to the lingering floating approval. Deposits fail when attempting to approve in the `getTokensForPool` function, and withdrawals fail when trying to grant new approval in the `withdraw` function.

RECOMMENDATION

Do not grant approvals when they are not needed. Alternatively, consider checking the token address and clearing previous approvals if the token is USDT.

RESOLUTION

Status: Resolved.

<https://github.com/MEWB-AS/vanir-contracts/pull/12/commits/36f27f384cd4e4d520061394b83e60382c04545f>

LOW **L-01**

RESOLVED

State Variables Missing Immutable Modifier

CATEGORY Optimization
LOCATION Vanir.sol
REVIEW Main Review

DESCRIPTION

The `Vanir` contract declares several state variables that are assigned values only once during contract deployment and are not intended to change thereafter. Specifically, the variables `flash`, `poolAddressProvider`, and `owner` are set in the constructor and never modified again throughout the contract's lifecycle. However, these variables are not marked with the `immutable` keyword. In Solidity, variables that are assigned once during construction and remain constant can be declared as `immutable`, which offers gas savings by storing them directly in the bytecode rather than in storage. By not marking these variables as `immutable`, the contract incurs unnecessary gas costs for storage reads when accessing them.

RECOMMENDATION

Consider declaring the mentioned state variables as `immutable`.

RESOLUTION

Status: Resolved.

<https://github.com/MEWB-AS/vanir-contracts/pull/12/commits/cf2a92b51dd0ae470f2f84fb6a075f466b5676b0>

LOW **L-02**

RESOLVED

getUserAccountData Function Can Be Declared As View

CATEGORY Optimization
LOCATION Vanir.sol: 52
REVIEW Main Review

DESCRIPTION

The `getUserAccountData` function in the `Vanir` contract is currently not declared with the `view` modifier, even though it does not modify any state variables. This function simply retrieves user account data from the lending pool by calling `getPool().getUserAccountData(user)` and returns the results. In Solidity, functions that do not alter the state of the contract should be marked as `view`. The absence of the `view` modifier leads to unnecessary gas costs as the EVM will treat the function as a state-changing transaction rather than a simple call.

RECOMMENDATION

Consider declaring the `getUserAccountData` as a `view` function.

RESOLUTION

Status: Resolved.

<https://github.com/MEWB-AS/vanir-contracts/pull/12/commits/36f27f384cd4e4d520061394b83e60382c04545f>

LOW **L-03**

RESOLVED

code Parameter Can Be Removed From The loan Function

CATEGORY Optimization
LOCATION Vanir.sol: 105
REVIEW Main Review

DESCRIPTION

In the `Vanir` contract, the `loan` function includes a parameter `code` of type `RequestCode`, which is an enum. This parameter is not utilized within the function's internal logic to influence any operations or decisions. Its sole usage is in the `emit Loan` statement at the end of the function, where it is included in the emitted event:

```
emit Loan(msg.sender, code, requests);
```

RECOMMENDATION

Consider removing the `code` parameter from both the `loan` function's parameters and the `Loan` event if it does not provide additional, necessary information that cannot be obtained from existing data like the `requests` array.

RESOLUTION

Status: Resolved.

<https://github.com/MEWB-AS/vanir-contracts/pull/12/commits/c79ee614728e6b45b3a58c7b7cd252c11b60d893>

LOW **L-04**

RESOLVED

USDS Liquidations Are Not Supported

CATEGORY Optimization
LOCATION Vanir.sol: 124
REVIEW Main Review

DESCRIPTION

The `Vanir` contract's liquidation functionality currently does not support liquidations for USDS, even though USDS is one of the tokens that can be borrowed in the Spark protocol alongside DAI. This limitation is due to the contract's design, which specifically handles DAI liquidations and lacks the flexibility to process other debt tokens. In the `liquidate` function, the parameter `debtTokenAddress` is intended to represent the debt token to be repaid, but the subsequent logic in `onFlashLoan` and related functions assumes that this token is always DAI. For example, hard-coded addresses and assumptions about token behavior are tailored to DAI, making the liquidation process incompatible with USDS or any other debt token.

RECOMMENDATION

Consider refactoring the `Vanir` contract to support liquidations for any debt token supported by the Spark protocol.

RESOLUTION

Status: Resolved.

<https://github.com/MEWB-AS/vanir-contracts/pull/12/commits/a7ded1a05d1cf6a2ad1316d8205ccabdd33defb7>

LOW **L-05**

RESOLVED

Liquidations Require A SPToken Approval To The Vanir Contract

CATEGORY Logical Error
LOCATION Vanir.sol: 148
REVIEW Main Review

— DESCRIPTION

The `Vanir` contract's liquidation process requires the user being liquidated to approve the contract to spend their `spToken` (Spark Protocol Token) collateral. Specifically, during liquidation, the contract calls the `withdraw` function, which attempts to transfer the user's `spTokens` to itself using `getTokensForPool`. This function checks the user's allowance and balance of `spTokens` and performs a `transferFrom` to move the tokens to the contract. However, this approach relies on the user's cooperation to grant approval to the `Vanir` contract for their `spTokens`. Without this approval, liquidations can not be performed.

— RECOMMENDATION

Merely an informative issue. There is no way to perform this process without the prior user approval.

— RESOLUTION

Status: Resolved.

LOW **L-06**

ACKNOWLEDGED

Use Of Transfer Instead Of Call

CATEGORY Configuration
LOCATION Vanir.sol: 90
REVIEW Main Review

DESCRIPTION

With the Solidity `transfer()` function, used to perform a transfer of a native asset, there is a gas limit of 2300 gas and the receiving smart contract should have a fallback function defined or else the transfer call will fail. This value is hardcoded to prevent reentrancy attacks.

In the `transferEth` function this `transfer` call is used to transfer the native assets. Some multi-signature wallets require more than 2300 gas to receive a native asset transfer. For example, Gnosis Safe supports forwarding via fallback. Using `transfer()` with a Safe's multisig can lead to gas depletion and failed transfers. Some references can be found below:

- [Article: why cant I transfer Ether from a contract into a safe](<https://help.safe.global/en/articles/40813-why-can-t-i-transfer-eth-from-a-contract-into-a-safe>)
- [Blog post: Stop using solidity's transfer now](<https://consensys.io/diligence/blog/2019/09/stop-using-soliditys-transfer-now/>)

RECOMMENDATION

It is recommended to use `call` over `transfer` to perform the transfer of a native asset. A proper gas limit should be set for the `call` making sure that is enough to cover transfers to any possible multi-signature wallets. Measures against reentrancy should be taken into consideration as well by for example using a `nonReentrant` lock.

RESOLUTION

Status: Acknowledged.

LOW **L-07**

RESOLVED

Owner State Variable Can Be Removed

CATEGORY Optimization
LOCATION Vanir.sol: 9
REVIEW Main Review

DESCRIPTION

The `Vanir` contract declares an `owner` state variable that is initialized in the constructor to the address of the contract deployer (`msg.sender`). However, throughout the contract code, the `owner` variable is never used in any function or modifier to control access or execute owner-specific operations. There are no references to `owner` beyond its initialization, and it does not influence the contract's behavior in any way.

RECOMMENDATION

Consider removing the `owner` state variable from the `Vanir` contract.

RESOLUTION

Status: Resolved.

<https://github.com/MEWB-AS/vanir-contracts/pull/12/commits/ae5d4e4ad99227083e2c63cc57458ff155b7bcfa>

LOW **L-08**

RESOLVED

Insecure setAdmin Function

CATEGORY Configuration
LOCATION Vanir.sol: 32
REVIEW Main Review

DESCRIPTION

The `Vanir` contract uses a `setAdmin` function to manage multiple admin addresses through an `admins` mapping. This admin role is critical because the admins are the only ones allowed to execute the `liquidate` function, which allows them to perform liquidations on user positions. If any admin account is compromised, they could abuse this privilege to drain user funds by providing harmful `LiquidationRequest` parameters or specifying a malicious `swapRouterAddress`.

RECOMMENDATION

Consider restricting the ability to add new admins exclusively to a single `owner` address. This `owner` should be managed by a multisig wallet. The `setAdmin` function should be updated so that only the `owner` can call it, preventing existing admins from granting admin rights to others. Finally, to implement secure ownership management, the contract should use [OpenZeppelin's](<https://github.com/OpenZeppelin/openzeppelin-contracts/blob/master/contracts/access/Ownable2Step.sol>) [`Ownable2Step`](<https://github.com/OpenZeppelin/openzeppelin-contracts/blob/master/contracts/access/Ownable2Step.sol>)[contract](<https://github.com/OpenZeppelin/openzeppelin-contracts/blob/master/contracts/access/Ownable2Step.sol>), which implements a two-step ownership transfer pattern. This pattern requires the new owner to accept the ownership preventing accidental changes to the `owner` address.

RESOLUTION

Status: Resolved.

<https://github.com/MEWB-AS/vanir-contracts/pull/12/commits/ae5d4e4ad99227083e2c63cc57458ff155b7bcfa>

LOW **L-09**

ACKNOWLEDGED

Unlicensed Smart Contract

CATEGORY Configuration
LOCATION Vanir.sol
REVIEW Main Review

— DESCRIPTION

The `Vanir` contract is currently marked as unlicensed, as indicated by the SPDX license identifier at the top of the file:

```
// SPDX-License-Identifier: UNLICENSED
```

Using unlicensed contracts can lead to legal uncertainties and conflicts regarding the usage, modification and distribution rights of the code.

— RECOMMENDATION

It is recommended to choose and apply an appropriate open-source license to the smart contract. Some options are:

1. MIT License: A permissive license that allows for reuse with minimal restrictions.
2. GNU General Public License (GPL): A copyleft license that ensures derivative works are also open-source.
3. Apache License 2.0: A permissive license that provides an express grant of patent rights from contributors to users.

— RESOLUTION

Status: Acknowledged.

LOW **L-10**

RESOLVED

Decreased Health Factor After Liquidation

CATEGORY Logical Error
LOCATION Vanir.sol: 152
REVIEW Main Review

— DESCRIPTION

Liquidations on Vanir are performed to prevent liquidations on the Spark platform. During the liquidation process, a portion of the user's collateral is withdrawn from Spark, swapped for the debt token to repay the flash loan, and any remaining collateral after the swap is refunded to the user. Ideally, a liquidation on Vanir should improve the user's health factor on Spark.

However, in the case of a partial liquidation, the user's health factor will decrease if the value of the withdrawn collateral exceeds the repaid debt. This occurs because the excess collateral is not re-supplied to Spark but instead transferred to the user, effectively acting as a collateral withdrawal on behalf of the user.

As a result, a liquidation on Vanir could potentially bring the user's position closer to liquidation on Spark, rather than preventing it.

— RECOMMENDATION

Re-supply the excess collateral to Spark on behalf of the user in cases of partial debt repayment.

PROOF OF CONCEPT <https://github.com/GuardianAudits/vanir-2/pull/2>

— RESOLUTION

Status: Resolved.

<https://github.com/MEWB-AS/vanir-contracts/pull/12/commits/fbf77b3e08873ca7f6c23135f8a922e9b858d03f>

LOW **L-11**

RESOLVED

Maximum Withdraw Is Not Supported

CATEGORY Configuration
LOCATION Vanir.sol
REVIEW Main Review

DESCRIPTION

The Spark pool's [withdraw()] (<https://etherscan.io/address/0x5ae329203e00f76891094dcfedd5aca082a50e1b#code#F8#L124>) function allows the passed amount to be `type(uint256).max`. This will be treated as the whole available balance of the given user. Vanir doesn't support this feature because if the amount to be withdrawn is greater than the user's balance, the transaction will revert.

RECOMMENDATION

Document this limitation.

RESOLUTION

Status: Resolved.

LOW **L-12**

RESOLVED

Unused fee Parameter

CATEGORY	Optimization
LOCATION	Vanir.sol
REVIEW	Main Review

DESCRIPTION

A `LoanRequests` consists of `Code` and `Asset`. The `Asset` has a `fee` parameter passed by the user which is never used in `Vanir.loan()`. However, it will be emitted in the `Loan` event as part of the `requests`.

RECOMMENDATION

Consider using different `Asset` structs for loan requests and liquidation requests.

RESOLUTION

Status: Resolved.

<https://github.com/MEWB-AS/vanir-contracts/pull/12/commits/cf2a92b51dd0ae470f2f84fb6a075f466b5676b0>

LOW **L-13**

ACKNOWLEDGED

Users Can Escape The Liquidation Fee

CATEGORY Configuration
LOCATION Vanir.sol
REVIEW Main Review

— DESCRIPTION

The idea of `Vanir.liquidate()` is to repay the debt of a given user by using their collateral before they get liquidated on `Spark`. By doing this a fee is to be charged by Vanir. However, users can just frontrun the `liquidate` transaction and repay their debt directly to avoid the `liquidate()` fee.

— RECOMMENDATION

Be aware of this issue.

— RESOLUTION

Status: Acknowledged.

LOW **L-14**

RESOLVED

Unnecessary Token Approval In withdraw

CATEGORY Logical Error
LOCATION Vanir.sol: 76
REVIEW Main Review

DESCRIPTION

When withdrawing, the contract is burning aTokens (spTokens) to receive the underlying token. The lending pool doesn't need approval for the underlying token.

The only approval needed is for the aTokens, which is already handled in `getTokensForPool`.

RECOMMENDATION

Remove the unnecessary approval.

RESOLUTION

Status: Resolved.

<https://github.com/MEWB-AS/vanir-contracts/pull/12/commits/36f27f384cd4e4d520061394b83e60382c04545f>

LOW **L-15**

RESOLVED

Unused fallback Function

CATEGORY	Optimization
LOCATION	Vanir.sol: 29
REVIEW	Main Review

— DESCRIPTION

The contract is expected to receive ETH through the WETH contract, which triggers the `receive` function. Therefore, the `fallback` function is unused and can be removed.

— RECOMMENDATION

Remove the `fallback` function.

— RESOLUTION

Status: Resolved.

<https://github.com/MEWB-AS/vanir-contracts/pull/12/commits/df2b07a6a2bf23ab8817b7d3634ae5a6350511cc>

LOW **L-16**

ACKNOWLEDGED

Referral Code Ommited

CATEGORY	Configuration
LOCATION	Vanir.sol: 59, 69, 96, 100, & 126
REVIEW	Main Review

— DESCRIPTION

Calls to Spark harcode zero for the referral code. Although they do not currently have referrals set up, they mention in their documentation that they may add them in the future, Since the referral is hardcoded to zero, Vanir will not be eligible for the referral program in the future.

— RECOMMENDATION

Use a variable, that can be updated in the future, to represent the referral code.

— RESOLUTION

Status: Acknowledged.

LOW **L-17**

RESOLVED

setAdmin() Should Be external

CATEGORY Optimization
LOCATION Vanir.sol: 32
REVIEW Main Review

— DESCRIPTION

`setAdmin()` is marked as a `public` function. However, `setAdmin()` is not called at all within the smart contract. Therefore, it can be set to an `external` function, which is more gas efficient.

— RECOMMENDATION

Change the function from `public` to `external`.

— RESOLUTION

Status: Resolved.

<https://github.com/MEWB-AS/vanir-contracts/pull/12/commits/cab8435af196eecaab2583c4da607f1995c6254>

LOW **L-18**

ACKNOWLEDGED

Lack Of Reentrancy Guard Modifier

CATEGORY	Configuration
LOCATION	Global
REVIEW	Main Review

— DESCRIPTION

There are numerous external calls throughout the protocol that hand over the execution flow to smart contracts outside of the Vanir protocol. It is recommended to use a reentrancy guard to prevent any potential reentrancy attacks.

— RECOMMENDATION

Use a reentrancy modifier on functions to prevent potential reentrancy attacks.

— RESOLUTION

Status: Acknowledged.

LOW **L-19**

RESOLVED

Check Debt Amounts During liquidate

CATEGORY Validation
LOCATION Vanir.sol: 124-127
REVIEW Main Review

DESCRIPTION

The `liquidate` function is called by the admins and used to liquidate some of the collateral of the user. The function accepts a `LiquidationRequest[] requests` array and every request has a `debtAmount`. The function also accepts another `totalDebtAmount` value, which will be flash-loaned.

However, there is no validation to ensure that the sum of all `requests.debtAmount` values equals `totalDebtAmount`. If `totalDebtAmount` exceeds the sum of `requests.debtAmount`, the flash loan cannot be repaid. Conversely, in the opposite scenario, any excess funds swapped will be locked in the contract since there isn't a rescue function.

RECOMMENDATION

Consider validating debt amounts when calling `liquidate`.

RESOLUTION

Status: Resolved.

<https://github.com/MEWB-AS/vanir-contracts/pull/12/commits/a7ded1a05d1cf6a2ad1316d8205ccabdd33defb7>

LOW **L-20**

ACKNOWLEDGED

Unvalidated Token Addresses

CATEGORY	Validation
LOCATION	Global
REVIEW	Main Review

— DESCRIPTION

During Vanir actions, token addresses are not validated. A user could set `isEth` to true while providing a token address other than `WETH`. Although such actions would be caught on the Spark side, it is best to validate token addresses within the Vanir contract, as it makes unsafe external calls to user-provided addresses.

— RECOMMENDATION

Validate token addresses, particularly when `isEth` is true.

— RESOLUTION

Status: Acknowledged.

CRITICAL C-01

RESOLVED

Unvalidated Parameters Allow an Attacker to Drain User Funds

CATEGORY	Validation
LOCATION	Vanir.sol
REVIEW	Remediation Review 1

DESCRIPTION

The `executeOperation` function permits callers to specify arbitrary parameters that will be used in different operations. This blind trust in user provided parameters allows the following exploit:

- Alice approves the Vanir contract with 10 WETH.
- Bob sees the approval and calls from a malicious contract address `executeOperation(<WETH>, 10e18, 0, <Vanir address>, <params>)` where `params` will be decoded to:

```
address user = Alice
address lendingPoolAddress = < Malicious address deployed by Bob >
collateralAsset.amount = 10e18
collateralAsset.token = WETH
collateralAsset.data = data that will be decoded to
address spTokenAddress = WETH
uint feeAmount = 0
address swapRouterAddress = < Malicious address deployed by Bob >
address feeAddress = address(0)
bytes memory swapPath = ''
```

The following code will be executed:

```
// In executeOperation
// Check below will pass because we will call this function from the lendingPoolAddress
contract
require(msg.sender == address(lendingPoolAddress), "vanir/flash loan not opened by lending
pool.");
// Initiator is controlled by Bob and was set to Vanir's address
require(initiator == address(this), "vanir/flash loan must be initiated by Vanir.");
// Will WETH approve Bob's malicious contract address
Token(asset).approve(lendingPoolAddress, amount);
```

C-01 Unvalidated Parameters Allow an Attacker to Drain User Funds

CONTINUED

```
// Call to Bob' malicious contract address. Repay might do nothing in Bob's malicious contract
address
ILendingPool(lendingPoolAddress).repay(asset, amount, 2, user); // only variable rate mode 2
supported
// processLiquidation is called, lets continue the flow
uint swapCost = processLiquidation(user, asset, amount, lendingPoolAddress, collateralAsset);
// processLiquidation flow
/*
The parameters below will be decoded to as they are controlled by Bob:
address spTokenAddress = WETH
uint feeAmount = 0
address swapRouterAddress = < Malicious address deployed by Bob >
address feeAddress = address(0)
bytes memory swapPath = ''
*/
(address spTokenAddress, uint feeAmount, address swapRouterAddress, address feeAddress, bytes
memory swapPath) = abi.decode(collateralAsset.data, (address, uint, address, address, bytes));
/*
// Withdraw flow:
// This will transfer the 10 WETH from Alice to Vanir address
Token(spTokenAddress).transferFrom(from, address(this), amount);
// The withdraw function will pull the 10 WETH from Vanir's address to the malicious contract
address owned by Bob as it was previously approved
ILendingPool(lendingPoolAddress).withdraw(tokenAddress, amount, to);
*/
withdraw(user, address(this), collateralAsset.amount, collateralAsset.tokenAddress,
spTokenAddress, lendingPoolAddress);
// Bob already stole the WETH so the rest of the functions can simply be empty and return 0
values
swapCost = swap(swapRouterAddress, collateralAsset.amount, flashLoanAmount + feeAmount,
collateralAsset.tokenAddress, swapPath);
if (feeAmount > 0) {
Token(debtTokenAddress).transfer(feeAddress, feeAmount);
}
uint supplyAmount = collateralAsset.amount - swapCost;
if (supplyAmount > 0) {
Token(collateralAsset.tokenAddress).approve(lendingPoolAddress, supplyAmount);
```

C-01 Unvalidated Parameters Allow an Attacker to Drain User Funds

CONTINUED

```
ILendingPool(lendingPoolAddress).supply(collateralAsset.tokenAddress, supplyAmount, user, 0);  
}  
return swapCost;  
// Back to executeOperation flow  
Token(asset).approve(lendingPoolAddress, amount + premium);  
emit Liquidation(user, collateralAsset, swapCost, asset, amount, premium);
```

This way Bob managed to steal Vanir approved funds from Alice. Anytime a contract will call `transferFrom(user, ...)` based on an address passed in by an external call, you can end up pulling the user's real tokens if they have an allowance. The contract must enforce that the `lendingPoolAddress` is truly whitelisted and not malicious.

— RECOMMENDATION

Ensure that the `lendingPoolAddress` and the `swapRouterAddress` provided are whitelisted by the `Vanir` contract.

PROOF OF CONCEPT <https://github.com/GuardianAudits/vanir-1/pull/4>

— RESOLUTION

Status: Resolved.

<https://github.com/MEWB-AS/vanir-contracts/pull/14/commits/5228db09ac53627261ee3ddcc1cea7ede1942981>

HIGH **H-01**

RESOLVED

Unrestricted Direct Contract Interaction

CATEGORY	Validation
LOCATION	Vanir.sol
REVIEW	Remediation Review 1

DESCRIPTION

The `Vanir` contract is designed to work primarily via a centralized frontend. The expectation is that the backend logic will always supply “safe” parameters such as a legitimate `lendingPoolAddress`, `swapRouterAddress`, `feeAddress`, `feeAmount`, etc. However, there is no on-chain restriction preventing a user from calling the `Vanir` contract directly providing arbitrary parameters.

Therefore, malicious users can pass a malicious contract as `lendingPoolAddress` or `swapRouterAddress`. The contract logic (`executeOperation`, `processLiquidation`, `swap`, etc.) will trust these addresses to perform critical actions like repaying debt, withdrawing collateral, and swapping tokens.

Also, the contract allows specifying a custom `feeAmount` and `feeAddress` in actions like `setup` or `interest`. An attacker who directly calls the contract can simply set the `feeAmount` to zero to avoid paying any fee.

RECOMMENDATION

If the design truly requires a centralized backend to set those parameters, enforce a server-signed EIP-712 message containing the parameters. The smart contract would verify the signature before processing. This ensures malicious direct calls without a valid signature revert.

RESOLUTION

Status: Resolved.

<https://github.com/MEWB-AS/vanir-contracts/pull/14/commits/5228db09ac53627261ee3ddcc1cea7ede1942981>

HIGH **H-02**

RESOLVED

Liquidations Will Revert When Flashloan Premium Is Different Than 0

CATEGORY Logical Error
LOCATION Vanir.sol
REVIEW Remediation Review 1

DESCRIPTION

After a flashloan is initiated, AAVE performs a callback through the `executeOperation` function passing as parameters the loan `amount` and `premium` which are the fees to be paid.

However, in Vanir's `executeOperation`, only `amount` is passed to `processLiquidation`. As a result, there won't be sufficient loan token swapped to repay the flashloan, and the entire liquidation process will fail.

RECOMMENDATION

Add `premium` to `amount` when calling the `processLiquidation` function:

```
uint swapCost = processLiquidation(user, asset, amount + premium, lendingPoolAddress, collateralAsset);
```

RESOLUTION

Status: Resolved.

<https://github.com/MEWB-AS/vanir-contracts/pull/14/commits/194d18fa6154f1ac392413ffa243ab7c516ae4f8>

MEDIUM **M-01**

RESOLVED

USDT swap is blocked

CATEGORY DoS
LOCATION Vanir.sol: 202
REVIEW Remediation Review 1

DESCRIPTION

In the `swap()` function Vanir approves the `swapRouterAddress` to spend `amountIn` tokens. It's expected that in most cases the router won't use up all of these tokens. Because the approval is not reset to 0, the difference between the originally approved amount and the actual spent amount will be left approved. This will cause DOS of the swap function for the `USDT` token which doesn't support changing allowances from a non-zero value to another non-zero value.

RECOMMENDATION

Consider clearing the approval to the `swapRouterAddress` after the swap concludes.

RESOLUTION

Status: Resolved.

<https://github.com/MEWB-AS/vanir-contracts/pull/14/commits/7ae686114dbf0ee9f4b7099efdea84a7cae0701e>

LOW **L-01**

RESOLVED

Owner cannot be admin

CATEGORY Code Best Practices
LOCATION Vanir.sol: 29
REVIEW Remediation Review 1

DESCRIPTION

The `setAdmin()` function doesn't allow the owner of the Vanir contract to be an owner as well. This can be inconvenient since you need 2 addresses for 1 admin.

RECOMMENDATION

You can change the require statement to:

```
require(flag || msg.sender != admin, "vanir/owner cannot remove its own admin status.")
```

This will allow the owner to add themselves as an admin, but not remove themselves.

RESOLUTION

Status: Resolved.

LOW **L-02**

RESOLVED

Typo

CATEGORY Code Best Practices
LOCATION Vanir.sol: 29
REVIEW Remediation Review 1

DESCRIPTION

The error message inside `Vanir.setAdmin()` is "vanir/owner cannot change its own adminstatus.". There is no spacing between `admin` and `status`.

RECOMMENDATION

Add a spacing between the two words `admin` and `status`.

RESOLUTION

Status: Resolved.

LOW **L-03**

ACKNOWLEDGED

Token And spToken Can Mismatch

CATEGORY Validation
LOCATION Vanir.sol: 130-136
REVIEW Remediation Review 1

DESCRIPTION

During the withdrawal process, both `tokenAddress` and `spTokenAddress` are provided by the user. There is no check to ensure that these addresses match each other as their respective pairs on the underlying Spark protocol.

RECOMMENDATION

Check that `tokenAddress` and `spTokenAddress` are corresponding addresses.

RESOLUTION

Status: Acknowledged.

LOW **L-04**

ACKNOWLEDGED

Validate Asset Is Not Used Twice

CATEGORY Validation
LOCATION Vanir.sol: 110
REVIEW Remediation Review 1

— DESCRIPTION

M-01 was acknowledged as intended behaviour, but the function `loan()` has no access restriction and can be called by any user. This can lead to the issue from M-01, where the function reverts with multiple interactions with ether in the same call.

— RECOMMENDATION

Clearly document that `loan()` is intended to not handle the same asset twice in one function call. Additionally, consider adding validation to prevent users from calling `loan()` with multiple occurrences of the same asset.

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-01**

ACKNOWLEDGED

User Can Pass Arbitrary Fee Amount

CATEGORY Logical Error
LOCATION Vanir.sol
REVIEW Remediation Review 2

DESCRIPTION

Function `borrow` accepts an arbitrary `feeAmount` through a `loan` call. If Vanir expects a fee to be paid on borrow, a user can just directly interact with the contract and pass a `feeAmount` of zero to preserve as much capital.

RECOMMENDATION

Add validation on the expected `feeAmount` for Vanir. Alternatively, consider tying `feeAmount` to a fixed percentage of the borrowed/liquidated amount rather than letting the caller supply an arbitrary fee, implement a fixed percentage model (e.g., $\text{feeAmount} = (\text{borrowedAmount} * \text{feePercentage}) / 1e18$).

RESOLUTION

Status: Acknowledged.

MEDIUM **M-02**

RESOLVED

Failed Approvals With USDT

CATEGORY Logical Error
LOCATION Vanir.sol
REVIEW Remediation Review 2

DESCRIPTION

Function `repay` approves `amount` and then calls SparkLend to repay the amount. The issue is that the `paybackAmount` within the BorrowLogic is not necessarily equal to `amount` passed, so a portion of the approved amount will be not be utilized.

This will lead to DoS with tokens such as USDT which require a 0 approval initially.

RECOMMENDATION

For all allowances in Vanir, use SafeERC20's `forceApprove` which will force the allowance to go to zero initially to handle tokens such as USDT. Also, use it after an external call to set the approval to zero after the approval is no longer necessary.

PROOF OF CONCEPT <https://github.com/MEWB-AS/vanir-contracts/pull/15/commits/2acb753a9c37501ff406b4e569fd02ce35f9ef16>

RESOLUTION

Status: Resolved.

MEDIUM **M-03**

ACKNOWLEDGED

Lost borrow fee

CATEGORY Validation
LOCATION Vanir.sol
REVIEW Remediation Review 2

DESCRIPTION

When users borrow via Vanir, a part of the borrowed assets is paid as a fee to the `feeAddress`, which is an arbitrary passed parameter, but is validated to be a whitelisted address.

However, whitelisted addresses are also the swap routers and the lending pools. This means users may choose to pay the fee to any other whitelisted address resulting in loss of funds for Vanir.

RECOMMENDATION

Create a separate validation for the `feeAddress`.

RESOLUTION

Status: Acknowledged.

LOW **L-01**

RESOLVED

LendingPoolAddress Points To Registry Instead

CATEGORY Deployment
LOCATION DeployVanir.s.sol
REVIEW Remediation Review 2

DESCRIPTION

The deployment script currently passes `0x02C3eA4e34C0cBd694D2adFa2c690EECbC1793eE` (the `PoolAddressesProvider` of Spark) to the `Vanir` constructor, instead of passing `0xC13e21B648A5Ee794902342038FF3aDAB66BE987` (the Spark `LendingPool` address on Ethereum mainnet). As a result, the `Vanir` contract is initialized with the wrong contract reference thereby preventing it from interacting correctly with the Spark lending protocol.

If the contract is meant to integrate with Spark Lend on mainnet, the `LendingPool` address (`0xC13e21B648A5Ee794902342038FF3aDAB66BE987`) should replace the `PoolAddressesProvider` address in the script. Using the `PoolAddressesProvider` directly is incorrect in this context because `Vanir` specifically needs the active `LendingPool` to perform supply/borrow/repay operations, not just an address provider.

RECOMMENDATION

Update the constructor call in `DeployVanir` (or any relevant deployment script) to use `0xC13e21B648A5Ee794902342038FF3aDAB66BE987` instead of `0x02C3eA4e34C0cBd694D2adFa2c690EECbC1793eE`.

PROOF OF CONCEPT <https://github.com/MEWB-AS/vanir-contracts/pull/15/commits/3e0791137a3607cc4fbb89017a6190f9ea0104fe>

RESOLUTION

Status: Resolved.

LOW **L-02**

RESOLVED

No Controls On Liquidation Fee Amount

CATEGORY Validation
LOCATION Vanir.sol
REVIEW Remediation Review 2

DESCRIPTION

During liquidation, the user pays an additional `feeAmount` from their collateral. This `feeAmount` is arbitrarily set by the admin within their `LiquidationRequest`, and can vary each time. Without fee validation, the fee may be too small or too large, negatively impacting the protocol and user respectively.

RECOMMENDATION

Consider adding on-chain validations for `feeAmount` on liquidations, or clearly document the fee calculation behavior of your backend system.

RESOLUTION

Status: Resolved.

LOW **L-03**

RESOLVED

Failed Liquidations With Low Target LTV

CATEGORY	Warning
LOCATION	Global
REVIEW	Remediation Review 2

— DESCRIPTION

Vanir has a target LTV for liquidation within its backend system. If the LTV which Vanir is targetting has a large delta between the current LTV, all liquidation calls will fail as more amountIn is necessary than is approved and available for swap.

— RECOMMENDATION

Clearly document this behavior and appropriately set the target LTV.

— RESOLUTION

Status: Resolved.

LOW **L-04**

ACKNOWLEDGED

Failing Tests

CATEGORY	Warning
LOCATION	Global
REVIEW	Remediation Review 2

— DESCRIPTION

The tests are meant to simulate Vanir's operations with current backend configurations, however numerous tests are failing. Some reasons for failure include but are not limited to:

1. Lack of pinned fork block number
2. Improper Target LTV's for partial liquidations
3. Incorrect feeAmount calculation within the tests

All tests should be passing prior to deployment.

— RECOMMENDATION

Fix all tests and align them with Vanir's backend systems.

— RESOLUTION

Status: Acknowledged.

LOW **L-05**

ACKNOWLEDGED

No events emitted on funds rescue

CATEGORY	Event
LOCATION	Vanir.sol
REVIEW	Remediation Review 2

— DESCRIPTION

Two new functions has been added to the contract to enable the admin to resuce funds - 'transfer' and 'transferEth'. There are no events emitted to notify for the action happening.

— RECOMMENDATION

Consider if you should emit events in these two functions.

— RESOLUTION

Status: Acknowledged.

LOW **L-06**

ACKNOWLEDGED

Frozen Pool Will Lead To Fail Liquidations

CATEGORY Warning
LOCATION Global
REVIEW Remediation Review 2

— DESCRIPTION

When the reserve configuration in SparkLend `isFrozen`, supplying liquidity is prevented by repays and withdraws are permitted. If the configuration were set to frozen, most Vanir liquidations would fail since function `swap()` attempts to supply liquidity that wasn't used as part of the swap cost.

— RECOMMENDATION

Clearly document this risk.

— RESOLUTION

Status: Acknowledged.

LOW **L-01**

ACKNOWLEDGED

Incorrected Addresses

CATEGORY Warning
LOCATION deploy-tenderly.json
REVIEW Remediation Review 3

DESCRIPTION

File `deploy-tenderly.json` has an incorrect `lendingPool` address (currently PoolAddressProvider) and `feesAddress` (currently the SparkLend Pool).

RECOMMENDATION

Update the addresses to `0xC13e21B648A5Ee794902342038FF3aDAB66BE987` and `0xc7453Ea3051eE01639eF13D67BF7d6a077ee77Dc` respectively.

RESOLUTION

Status: Acknowledged.

LOW **L-02**

ACKNOWLEDGED

Leftover SwapRouter Approval

CATEGORY Warning
LOCATION Vanir.sol
REVIEW Remediation Review 3

DESCRIPTION

After function `swap` is performed, the SwapRouter may still have some token approval leftover if the entire `amountIn` was not utilized.

RECOMMENDATION

Remove the leftover approval after the swap.

RESOLUTION

Status: Acknowledged.

LOW **L-03**

ACKNOWLEDGED

Overcharge in executeOperation

CATEGORY	Logical Error
LOCATION	Vanir.sol
REVIEW	Remediation Review 3

DESCRIPTION

As we know from the previous round, the `SparkLend.repay()` function may not use all of the tokens we pass as `amount`. In `executeOperation` the contract repays with the flashloaned tokens, withdraws collateral and swaps it for the flashloaned amount + premium. If the `repay()` function didn't use all of the flashloaned amount, the function will be overcharging the user.

RECOMMENDATION

Be aware and clearly document this behavior to users.

RESOLUTION

Status: Acknowledged.

§ 05

Disclaimer.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian's position is that each company and individual are responsible for their own due diligence and continuous security. Guardian's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis.

Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract's safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

§ 06

About Guardian.

Guardian is a smart contract security firm. Every engagement is worked by two independent teams competing on the same scope, each with its own tooling and methods, so no single reviewer's blind spot survives.

\$45B+DIGITAL ASSETS UNDER
REVIEW**200+**

ENGAGEMENTS COMPLETED

500+CRITICAL & HIGH FINDINGS
REPORTED

Reach us at audits@guardianaudits.com.